



Harbinger: Dataset Examples



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Outline

This collection presents the benchmarks distributed through Harbinger's unified integration layer. The goal is practical: understand what each benchmark contains **before** using it in anomaly detection, change-point, or motif discovery experiments.

01

Benchmark Files

Yahoo S5, NAB, and UCR — widely used reference collections for time series analysis

02

Domain Collections

MIT-BIH, 3W Oil, and GECCO — real-world signals with rich applied context

03

Recommended Workflow

First understand the collection structure, then select the appropriate family of techniques

📄 Run the full examples and explore the source code at: <https://github.com/cefet-rj-dal/harbinger/tree/master/examples>

Inspect Before You Model

Before applying any event detection technique to a time series dataset, it's essential to understand its underlying structure. This "inspect first" discipline prevents mismatches between the dataset's characteristics and the chosen detection method.

01

Dataset Structure

In Harbinger, a dataset is a list of data frames, each representing one time series.

02

Column Separation

Metadata columns (`idx`, `event`, `type`, `seq`, `seqlen`) are separated from the actual signal columns.

03

Data Type

Classify as univariate or multivariate based on the number of signal columns present in the data.

04

Visualize First

Always perform a visual inspection (plot preview) of your time series data before proceeding with modeling.

 For a comprehensive treatment of time series event detection, see: Ogasawara, E., Salles, R., Porto, F., Pacitti, E. Event Detection in Time Series. Springer, 2025. doi:10.1007/978-3-031-75941-3

Dataset Inspection Pattern

```
library(harbinger)

dataset_summary <- function(x) {
  first_series <- x[[1]]
  meta_cols <- c("idx", "event",
    "type", "seq", "seqlen")
  signal_cols <- setdiff(
    names(first_series), meta_cols)
  dataset_type <- if (
    "value" %in% names(first_series) ||
    length(signal_cols) == 1
  ) "univariate" else "multivariate"
  plot_column <- if (
    "value" %in% names(first_series)
  ) "value" else signal_cols[1]
  list(
    n_series = length(x),
    dataset_type = dataset_type,
    signal_cols = signal_cols,
    plot_column = plot_column,
    preview_size = min(1000,
      nrow(first_series)),
    first_series = first_series
  )
}
```

```
show_dataset <- function(x, name) {
  info <- dataset_summary(x)
  cat("Dataset:", name, "\n")
  cat("Number of series:",
    info$n_series, "\n")
  cat("Dataset type:",
    info$dataset_type, "\n")
  invisible(info)
}

plot_dataset_preview <- function(info) {
  preview <- info$first_series[
    seq_len(info$preview_size),,
    drop = FALSE]
  har_plot(harbinger(),
    preview[[info$plot_column]],
    event = preview$event)
}
```

Step-by-Step Breakdown

- **meta_cols**: metadata columns excluded from signal analysis
- **signal_cols**: the remaining columns that carry actual signal values
- **dataset_type**: classified as univariate if a `value` column exists or only one signal column is present; otherwise multivariate
- **plot_column**: the first signal column selected for visualization
- **preview_size**: capped at 1,000 observations to keep plots readable and responsive
- **show_dataset()**: prints a concise summary to the console
- **plot_dataset_preview()**: calls `har_plot()` with the selected signal and its event labels

❏ **Key concept:** These three helper functions form a reusable inspection pattern used across all notebooks — load, summarize, and visualize before you model.

Yahoo S5 – A Labeled Benchmark for Anomaly Detection

The Yahoo S5 dataset is one of the most widely used benchmarks for time series anomaly detection, released by Yahoo Labs. It contains four sub-collections (A1–A4) that span both real-world and synthetic scenarios, providing a comprehensive testing ground for various detection techniques.

1 A1Benchmark 67 real-world production time series from Yahoo's infrastructure, with genuine anomaly labels from production incidents.	2 A2Benchmark 100 synthetic univariate series designed for controlled anomaly detection experiments.
3 A3Benchmark 100 synthetic series with labeled anomalous observations.	4 A4Benchmark 100 series combining point anomalies with structural change points, representing the most challenging scenarios.

All series within the Yahoo S5 benchmark are univariate, containing a single "value" column. The progression from A1 (real-world) to A4 (synthetic with complex change points) allows for systematic evaluation across different difficulty levels.

 Laptev, N., Amizadeh, S., Flint, I. (2015). Generic and scalable framework for automated time-series anomaly detection. In KDD '15.
[doi:10.1145/2783258.2788611](https://doi.org/10.1145/2783258.2788611)

A1Benchmark – Real-World Labeled Time Series

```
data(A1Benchmark)
A1Benchmark <- loadfulldata(A1Benchmark)
a1_info <- show_dataset(
  A1Benchmark, "A1Benchmark")

## Dataset: A1Benchmark
## Number of series: 67
## Dataset type: univariate
## Signals in the first series: value
## Column plotted with har_plot(): value
## Plot preview length: 1000 observations

plot_dataset_preview(a1_info)
```

Step-by-Step Breakdown

- **data()**: loads the lightweight stub object bundled with Harbinger
- **loadfulldata()**: replaces the stub with the complete dataset collection
- **show_dataset()**: confirms 67 univariate series, each with a `value` column
- **plot_dataset_preview()**: renders the first 1,000 observations alongside their anomaly labels

❏ **Key concept:** A1Benchmark draws directly from the Yahoo Webscope S5 dataset — its anomaly labels reflect genuine production incidents, making it an ideal benchmark for real-world detection tasks.

A2, A3 & A4 Benchmark – Synthetic Series

```
data(A2Benchmark)
A2Benchmark <-
loadfulldata(A2Benchmark)
a2_info <- show_dataset(
  A2Benchmark, "A2Benchmark")
## Number of series: 100
## Dataset type: univariate
```

```
data(A3Benchmark)
A3Benchmark <- loadfulldata(A3Benchmark)
a3_info <- show_dataset(
  A3Benchmark, "A3Benchmark")
## Number of series: 100
```

```
data(A4Benchmark)
A4Benchmark <- loadfulldata(A4Benchmark)
a4_info <- show_dataset(
  A4Benchmark, "A4Benchmark")
## Number of series: 100
```

```
plot_dataset_preview(a2_info)
plot_dataset_preview(a3_info)
plot_dataset_preview(a4_info)
```

Step-by-Step

- All three datasets follow the same workflow: `data()` → `loadfulldata()` → `show_dataset()` → `plot_dataset_preview()`
- **A2**: 100 synthetic series designed for anomaly detection benchmarking
- **A3**: 100 synthetic Yahoo series with labeled anomalous observations
- **A4**: 100 series combining synthetic anomalies with structural change points

 **Key concept:** A4 is the only Yahoo benchmark that blends point anomalies with change points — making it ideal for evaluating detectors that handle multiple event types.

NAB – Streaming Anomaly Detection in Real Time

The Numenta Anomaly Benchmark (NAB) was created to evaluate anomaly detection algorithms in a streaming, real-time context, fundamentally different from offline, batch-oriented benchmarks. Its unique scoring system penalizes late detections and rewards early ones, reflecting the operational cost of delayed alerts.

Synthetic & Real-World Series

NAB includes diverse sub-collections from artificial data, ad exchange, AWS metrics, road traffic, and Twitter volume.

Event-Oriented Labeling

Detections within a specific scoring window around a true event are counted as correct, enabling flexible evaluation.

Real-Time Scoring

The scoring function explicitly penalizes late anomaly detections, making it ideal for real-time system evaluation.

Domain Diversity

Each sub-collection targets a distinct real-world context, such as infrastructure monitoring or social media trends.

All series within the NAB benchmark are univariate, simplifying algorithm development while providing rich, domain-diverse challenges.

 Lavin, A., Ahmad, S. (2015). Evaluating Real-Time Anomaly Detection Algorithms — The Numenta Anomaly Benchmark. In ICMLA 2015. [doi:10.1109/ICMLA.2015.141](https://doi.org/10.1109/ICMLA.2015.141)

NAB – Artificial & Real Ad Exchange

```
data(nab_artificialWithAnomaly)
nab_artificialWithAnomaly <-
  loadfulldata(nab_artificialWithAnomaly)
nab_artificial_info <- show_dataset(
  nab_artificialWithAnomaly,
  "nab_artificialWithAnomaly")
```

```
## Number of series: 5
## Dataset type: univariate
```

```
plot_dataset_preview(nab_artificial_info)
```

```
data(nab_realAdExchange)
nab_realAdExchange <-
  loadfulldata(nab_realAdExchange)
nab_adexchange_info <- show_dataset(
  nab_realAdExchange,
  "nab_realAdExchange")
```

```
## Number of series: 6
## Dataset type: univariate
```

```
plot_dataset_preview(nab_adexchange_info)
```

Step-by-Step

- **nab_artificialWithAnomaly:** 5 synthetic series with injected anomalies — ideal for controlled validation experiments
- **nab_realAdExchange:** 6 real-world ad traffic series — anomalies are event-driven and reflect genuine market behavior
- NAB uses event-oriented labeling: any detection within a window around the event is counted as correct
- The same inspection pattern applies throughout: load, summarize, and visualize

📄 **Key concept:** NAB evaluates detectors in a streaming, real-time setting — its scoring penalizes late detections, making it fundamentally different from benchmarks based on point-wise accuracy.

NAB – AWS, KnownCause, Traffic & Tweets

```
data(nab_realAWSCloudwatch)
nab_realAWSCloudwatch <-
  loadfulldata(nab_realAWSCloudwatch)
show_dataset(nab_realAWSCloudwatch,
  "nab_realAWSCloudwatch")
## Number of series: 6
```

```
data(nab_realKnownCause)
nab_realKnownCause <-
  loadfulldata(nab_realKnownCause)
show_dataset(nab_realKnownCause,
  "nab_realKnownCause")
## Number of series: 7
```

```
data(nab_realTraffic)
nab_realTraffic <-
  loadfulldata(nab_realTraffic)
show_dataset(nab_realTraffic,
  "nab_realTraffic")
## Number of series: 7
```

```
data(nab_realTweets)
nab_realTweets <-
  loadfulldata(nab_realTweets)
show_dataset(nab_realTweets,
  "nab_realTweets")
## Number of series: 10
```

Step-by-Step

- **nab_realAWSCloudwatch**: 6 series of cloud infrastructure metrics from AWS
- **nab_realKnownCause**: 7 series with a documented root cause — making anomalies easier to interpret and validate
- **nab_realTraffic**: 7 road traffic series featuring strong seasonal patterns
- **nab_realTweets**: 10 tweet-volume series — the largest subcollection in NAB
- All datasets are univariate; use `plot_dataset_preview()` to quickly inspect each series visually

❏ **Key concept:** Each NAB subcollection targets a distinct real-world domain. Choose the one that best reflects your problem context before comparing detection methods — domain alignment matters as much as algorithm choice.

UCR Anomaly Archive – Subsequence-Oriented Detection

The UCR Anomaly Archive is a curated collection of time series specifically designed for subsequence anomaly detection. Unlike point anomalies, UCR focuses on entire segments or patterns within the signal that deviate from the norm, a critical distinction for detector selection.

01

Diverse Domains

Includes ECG, NASA telemetry, physiological signals, and electrical power demand datasets.

02

Subsequence Anomalies

Anomalies are contiguous segments that show significant deviations in pattern or behavior.

03

Specialized Detectors

Requires sliding window-based or subsequence-aware algorithms, not simple point-level methods.

04

Compact & Representative

Each sub-collection features 10-11 series, offering focused yet varied challenges.

05

Cross-Domain Studies

Ideal for evaluating algorithms capable of generalizing across biomedical, aerospace, and energy sectors.

 Wu, R., Keogh, E. J. (2021). Current Time Series Anomaly Detection Benchmarks are Flawed and are Creating the Illusion of Progress. IEEE TKDE. [doi:10.1109/TKDE.2021.3112126](https://doi.org/10.1109/TKDE.2021.3112126)

UCR – ECG, NASA, Bleeding & Power Demand

```
data(ucr_ecg)
ucr_ecg <- loadfulldata(ucr_ecg)
ucr_ecg_info <- show_dataset(
  ucr_ecg, "ucr_ecg")
## Number of series: 10

data(ucr_nasa)
ucr_nasa <- loadfulldata(ucr_nasa)
ucr_nasa_info <- show_dataset(
  ucr_nasa, "ucr_nasa")
## Number of series: 11

plot_dataset_preview(ucr_ecg_info)
plot_dataset_preview(ucr_nasa_info)
```

```
data(ucr_int_bleeding)
ucr_int_bleeding <-
  loadfulldata(ucr_int_bleeding)
ucr_bleeding_info <- show_dataset(
  ucr_int_bleeding, "ucr_int_bleeding")
## Number of series: 10

data(ucr_power_demand)
ucr_power_demand <-
  loadfulldata(ucr_power_demand)
ucr_power_info <- show_dataset(
  ucr_power_demand, "ucr_power_demand")
## Number of series: 11

plot_dataset_preview(ucr_bleeding_info)
plot_dataset_preview(ucr_power_info)
```

Step-by-Step

- **ucr_ecg**: 10 ECG series — anomalies appear as atypical subsequences within the signal
- **ucr_nasa**: 11 series of NASA spacecraft telemetry data
- **ucr_int_bleeding**: 10 physiological signal series capturing internal bleeding events
- **ucr_power_demand**: 11 electrical demand series exhibiting strong periodic patterns
- UCR is **subsequence-oriented** — anomalies are segments, not isolated points; use `plot_dataset_preview()` for visual inspection

❏ **Key concept:** The UCR Anomaly Archive focuses on subsequence anomalies. Use sliding window-based detectors rather than point-level methods to get meaningful results.

MIT-BIH Arrhythmia – ECG-Based Event Detection

The **MIT-BIH Arrhythmia Database** is a cornerstone for evaluating event detection algorithms in biomedical signals. It comprises extensive ECG recordings, each organized by specific lead placements (electrode positions) which capture distinct electrical perspectives of the heart's activity.

ECG signals are inherently periodic, reflecting the regular rhythm of the heart. Anomalies within this dataset correspond to arrhythmic events, or irregular heartbeats, which are critical to detect in clinical applications. This database serves as a classic benchmark for robustly evaluating detectors on periodic, biomedical signals.

- The dataset is organized by lead: MLII (standard reference lead), V1, V2, and V5—each offering a different view of cardiac electrical activity.
- MLII is the recommended starting point as it provides the clearest signal for most arrhythmia types.
- All series are univariate with a single "value" column, simplifying analysis.
- Comparing detector results across different leads is a powerful way to validate the robustness of an algorithm against varying signal presentations.

 Moody, G. B., Mark, R. G. (2001). The impact of the MIT-BIH Arrhythmia Database. IEEE Engineering in Medicine and Biology Magazine, 20(3), 45–50. [doi:10.1109/51.932724](https://doi.org/10.1109/51.932724)

MIT-BIH Arrhythmia – ECG Leads

```
data(mit_bih_MLII)
mit_bih_MLII <- loadfulldata(mit_bih_MLII)
mit_mlii_info <- show_dataset(
  mit_bih_MLII, "mit_bih_MLII")
## Number of series: 5
## Dataset type: univariate

data(mit_bih_V1)
mit_bih_V1 <- loadfulldata(mit_bih_V1)
show_dataset(mit_bih_V1, "mit_bih_V1")
## Number of series: 5

plot_dataset_preview(mit_mlii_info)
```

```
data(mit_bih_V2)
mit_bih_V2 <- loadfulldata(mit_bih_V2)
show_dataset(mit_bih_V2, "mit_bih_V2")
## Number of series: 4

data(mit_bih_V5)
mit_bih_V5 <- loadfulldata(mit_bih_V5)
show_dataset(mit_bih_V5, "mit_bih_V5")
## Number of series: 5
```

Step-by-Step

- MIT-BIH objects are **organized by lead**: MLII, V1, V2, and V5 — each capturing a distinct electrical perspective of the heart
- **MLII**: the standard reference lead with 5 series — the recommended starting point for most analyses
- **V1 and V5**: 5 series each; **V2**: 4 series
- All datasets are univariate with a single value column
- Previews are capped at 1,000 observations to keep ECG traces readable and easy to interpret

📌 **Key concept:** Each lead captures a distinct electrical view of the heart — comparing detector results across leads is a powerful way to validate robustness and generalizability.

3W Oil-Well – Multivariate Industrial Fault Detection

The 3W dataset, released by Petrobras, offers real and simulated multivariate time series from oil well sensors. It's specifically designed for fault detection and classification in critical offshore oil production, serving as a vital benchmark for industrial safety applications.

- **Multivariate Signals:** Each series includes simultaneous readings from pressure (e.g., p_pdg, p_tpt), temperature (t_tpt, t_jus_ckp), and gas lift flow (qgl) sensors.
- **Fault-Type Organization:** Data is categorized by specific fault types (Types 1, 2, 4, 5, 6, 7, 8), each representing a unique operational failure mode.
- **Detector Requirements:** Necessitates detectors that can process and correlate multiple signals for effective anomaly identification.
- **Data Inconsistency:** Column naming conventions vary (uppercase vs. lowercase) across fault types; always verify column names before analysis.
- **Industrial Benchmark:** Provides a realistic testbed for evaluating multivariate anomaly and fault detection methods in demanding industrial settings.

 Vargas, R. E. V., Munaro, C. J., Ciarelli, P. M., et al. (2019). A realistic and public dataset with rare undesirable real events in oil wells. Journal of Petroleum Science and Engineering, 181, 106223. [doi:10.1016/j.petrol.2019.106223](https://doi.org/10.1016/j.petrol.2019.106223)

3W Oil-Well – Types 1, 2, 4, and 5

```
data(oil_3w_Type_1)
oil_3w_Type_1 <- loadfulldata(oil_3w_Type_1)
oil_type_1_info <- show_dataset(
  oil_3w_Type_1, "oil_3w_Type_1")
## Number of series: 5
## Dataset type: multivariate
## Signals: p_pdg, p_tpt, t_tpt,
## p_mon_ckp, t_jus_ckp,
## p_jus_ckgl, qgl
## Column plotted: p_pdg

data(oil_3w_Type_2)
oil_3w_Type_2 <- loadfulldata(oil_3w_Type_2)
show_dataset(oil_3w_Type_2,
"oil_3w_Type_2")
## Number of series: 6
```

```
data(oil_3w_Type_4)
oil_3w_Type_4 <- loadfulldata(oil_3w_Type_4)
show_dataset(oil_3w_Type_4, "oil_3w_Type_4")
## Number of series: 3
## Column plotted: p_jus_ckgl

data(oil_3w_Type_5)
oil_3w_Type_5 <- loadfulldata(oil_3w_Type_5)
show_dataset(oil_3w_Type_5, "oil_3w_Type_5")
## Number of series: 12

plot_dataset_preview(oil_type_1_info)
```

Step-by-Step

- All 3W types are **multivariate** — each series contains readings from multiple sensors simultaneously
- Typical sensors include: p_pdg, p_tpt, t_tpt, p_mon_ckp, t_jus_ckp, p_jus_ckgl, and qgl
- The plot_column defaults to the first available sensor — inspect the remaining signals manually for a complete picture
- Type 4 uses p_jus_ckgl as its leading column due to a different alphabetical ordering
- Type 5 contains 12 series, making it the largest subcollection in the 3W dataset

❏ **Key concept:** Multivariate datasets require an explicit column selection — dataset_summary() automatically identifies the first available sensor to get you started.

3W Oil-Well – Types 6, 7, and 8

```
data(oil_3w_Type_6)
oil_3w_Type_6 <- loadfulldata(oil_3w_Type_6)
oil_type_6_info <- show_dataset(
  oil_3w_Type_6, "oil_3w_Type_6")
## Number of series: 6
## Dataset type: multivariate
## Signals: P_PDG, P_TPT, T_TPT,
## P_MON_CKP, T_JUS_CKP,
## P_JUS_CKGL, QGL
## Column plotted: P_PDG
```

```
data(oil_3w_Type_7)
oil_3w_Type_7 <- loadfulldata(oil_3w_Type_7)
show_dataset(oil_3w_Type_7, "oil_3w_Type_7")
## Number of series: 4

data(oil_3w_Type_8)
oil_3w_Type_8 <- loadfulldata(oil_3w_Type_8)
show_dataset(oil_3w_Type_8, "oil_3w_Type_8")
## Number of series: 3
## Signals: p_pdg, p_tpt, t_tpt,
## p_mon_ckp, p_jus_ckgl, qgl

plot_dataset_preview(oil_type_6_info)
```

Step-by-Step

- **Types 6 and 7:** column names are in **uppercase** (P_PDG, P_TPT...) — pay close attention to case sensitivity in R
- **Type 8:** only 3 series and is missing the `t_jus_ckp` column, making its structure slightly different from the others
- Always inspect `signal_cols` before referencing any column by name to avoid runtime errors
- Type 7 contains just 4 series — the smallest sub-collection in the 3W dataset

📌 **Key concept:** Naming inconsistencies across types — such as uppercase vs. lowercase column names — are a common source of bugs. Always use `show_dataset()` to confirm exact column names before running any analysis.

GECCO – Water Quality Monitoring and Anomaly Detection

The GECCO dataset, originating from the GECCO Industrial Challenge 2018, features real-world water quality sensor data. It serves as a crucial benchmark for anomaly detection in environmental monitoring, a domain where anomalies signify actual contamination events with significant environmental implications.

- **Dataset Composition:** 10 univariate time series, each with a single "value" column representing a specific water quality parameter.
- **Real Anomalies:** Anomalies within the data correspond to genuine contamination events in drinking water supply systems.
- **Signal Characteristics:** Exhibits both predictable seasonal patterns and irregular, event-driven spikes, making it ideal for robust detector evaluation.
- **High Stakes:** Environmental monitoring faces high real-world costs for false negatives, as missed contamination events can have severe consequences.
- **Applied Context:** Though compact, the dataset offers a rich and practical context for developing and testing anomaly detection algorithms.

📘 Wever, M., Tornede, A., Mohr, F., Hüllermeier, E. (2018). LRML: Leveraging Reinforcement Learning for Online Metalearning. GECCO Industrial Challenge 2018. [doi:10.1145/3205651.3208238](https://doi.org/10.1145/3205651.3208238)

GECCO – Water Quality

```
data(gecco)
gecco <- loadfulldata(gecco)
gecco_info <- show_dataset(gecco, "gecco")
```

```
## Dataset: gecco
## Number of series: 10
## Dataset type: univariate
## Signals in the first series: value
## Column plotted with har_plot(): value
## Plot preview length: 1000 observations
```

```
# Visual check: confirms the signal structure
# and label arrangement before any
# modeling decisions are made
```

```
plot_dataset_preview(gecco_info)
```

Step by Step

- **data(gecco)**: loads the mini packaged object into memory
- **loadfulldata()**: expands it into the full collection of 10 time series
- **show_dataset()**: confirms the univariate structure with a single `value` column
- **plot_dataset_preview()**: a practical sanity check — displays the raw signal alongside its label layout so you can assess the data at a glance
- This preview is exactly what you need to decide whether the collection is well-suited for anomaly detection or event-based experiments

📌 **Key concept:** GECCO is a real-world water quality dataset from the GECCO Industrial Challenge 2018 — its anomalies represent actual contamination events, each carrying specific environmental meaning.

Collection Summary

Dataset	Series	Type	Domain	Key Characteristic
AlBenchmark	67	Univariate	Yahoo S5	Real labeled time series
A2 / A3 / A4	100 each	Univariate	Yahoo S5	Synthetic; A4 includes change-points
NAB (6 groups)	5–10	Univariate	Multiple	Real-time evaluation with event windows
UCR (4 groups)	10–11	Univariate	Multiple	Subsequence anomaly detection
MIT-BIH (4 leads)	4–5	Univariate	ECG	Arrhythmias segmented by lead
3W Oil (7 types)	3–12	Multivariate	Oil & Gas	7 sensors per series; distinct fault types
GECCO	10	Univariate	Water	Real-world water quality events

📄 **Reference:** Ogasawara, E., Salles, R., Porto, F., Pacitti, E. *Event Detection in Time Series*. Springer, 2025. doi:10.1007/978-3-031-75941-3

Additional Sources

Core Textbook

Ogasawara, E., Salles, R., Porto, F., Pacitti, E. *Event Detection in Time Series*. Springer, 2025.
doi:10.1007/978-3-031-75941-3

SoftED

Salles, R. et al. (2024). SoftED: Metrics for soft evaluation of time series event detection. *Computers and Industrial Engineering*. doi:10.1016/j.cie.2024.110728

Anomaly Detection Survey

Blázquez-García, A., Conde, A., Mori, U., Lozano, J. A. (2022). A review on outlier/anomaly detection in time series data. *ACM Computing Surveys*, 54(3), 1–33. doi:10.1145/3444690

Change-Point Detection

Killick, R., Fearnhead, P., Eckley, I. A. (2012). Optimal detection of changepoints with linear cost. *JASA*, 107(500), 1590–1598.

UniTED

Lima, J., Castro, H., Oliveira, L., Paixão, E., Baroni, L., Salles, R., Vargas, R., and Ogasawara, E. "UniTED: A Unified Time Series Event Detection Repository." In *Brazilian e-Science Workshop (BreSci)*, SBC, Sept. 2025, pp. 1–8. doi:10.5753/bresci.2025.247972

Harbinger on GitHub

Explore examples, open issues, contribute, and follow development on the official GitHub repository:
github.com/cefet-rj-dal/harbinger